

GESTÃO DE REDES / NETWORK MANAGEMENT
Notas complementares / Complementary notes

Network Monitoring

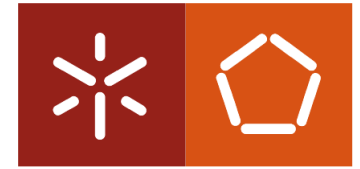
Remote Monitoring MIB





- **Terminology:**
 - *Measurements*
 - *Metrics*
- **Why are measurements needed?**
 - Network design and good capacity planning
 - Fault detection, finding anomalies
 - Verifying SLAs (Service Level Agreements)
 - Monitoring QoS
 - Validating Policies, prices, etc..
- **What can we measure?**
 - Latency, throughput, connectivity, packet loss, periodicity, ...

Introduction



- **Types of measures**

- **Passive**

- use devices to “observe” traffic and collect data
 - But: It may not be possible to “observe” all traffic → **SAMPLING**

- **Active**

- depends on dedicated data stream, generated by measuring service
 - But: It increases traffic and modifies environment

- **Hybrid**

- Uses elements of active and passive measurement



- **Measurement overhead**

- It may not be possible to measure everything!
- Need to transport data: measured in one place, processed in other
- Large number of measurement points, small number of aggregators (databases, summarized data)
- Information processing is not the bottleneck

- **Methods to reduce overhead**

- *Filtering*
- *Aggregation*
- *Sampling*
- Combinations of the those three...



Introduction

- **Filtering:**

- Only record packets that match filter criteria (ex: matching any field from header, or combination of fields)

- **Aggregation:**

- Use a criteria for aggregation of values
- Example: per flow statistics, identified by source & destination pair

Source	Destination	#bytes	#packts	#errors
a.b.c.d	x.y.z.k	25001	304	0
...	...	...	...	...

- Problem with table size: can be solved by using a **fixed cache size**
 - Policy: LRU, ? Others...



Introduction

- **Sampling:**
 - *Systematic sampling*
 - ex: pick every 100th packet ... record header
 - Problem if there are periodic components (Why?)
 - *Random sampling*
 - Capture with probability P previously established
 - Flip a coin on every packet
 - *Dynamic sampling*
 - Try to adjust the probability according to traffic patterns
- **Sampling may not be a easy task (not addressed here!)**

Introduction



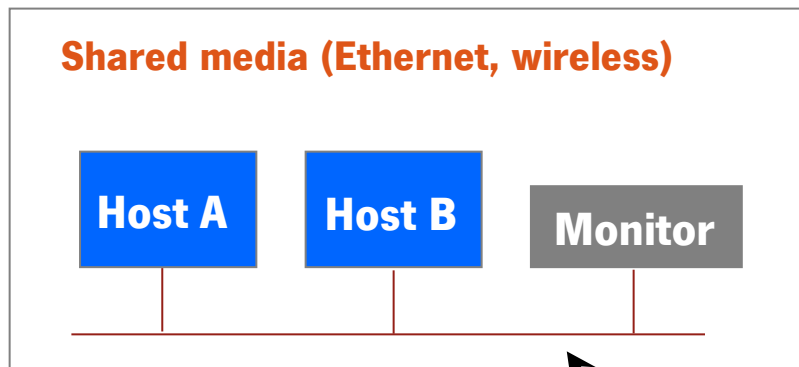
- **Methods compared**

	Filtering	Aggregation	Sampling
Precision	exact	exact	approximate
Generality	Constrained a-priori	Constrained a-priori	general
Local Processing	Filter criterion for every object	Table update for every object	Only sampling decision
Local memory	none	One bin per value of interest	none
Compression	Depends on data	Depends on data	Controlled

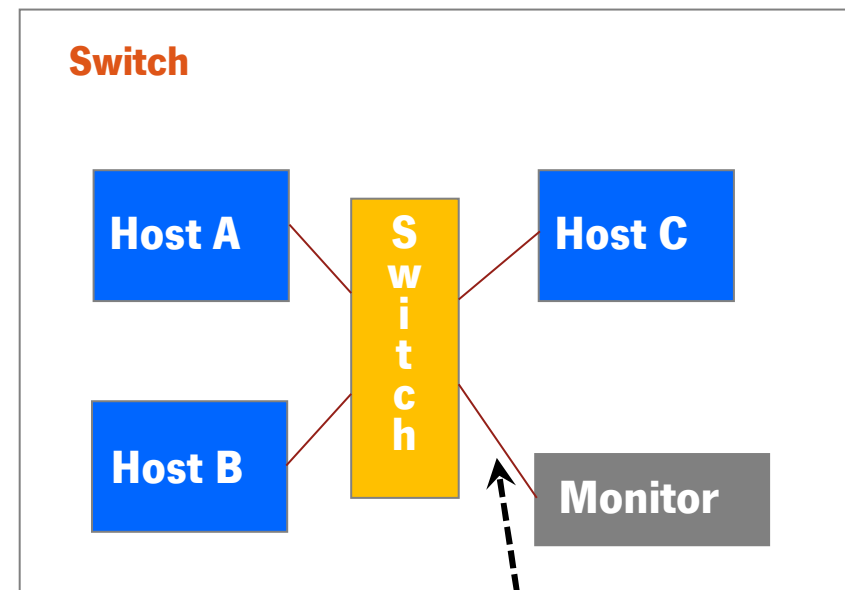


Introduction

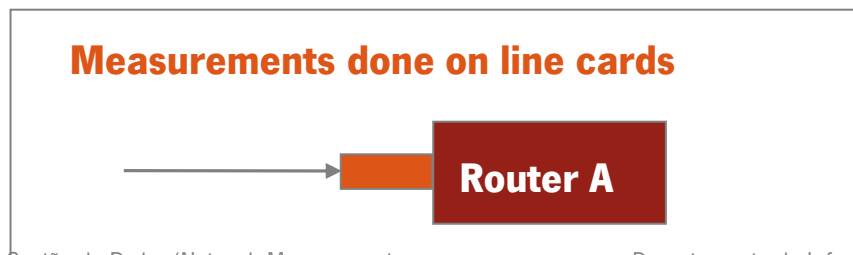
- **Devices and passive measurements**



*Traffic can be captured
in shared media
(promiscuous mode)*



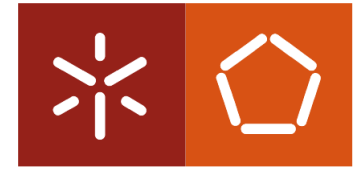
*On switch port configured
as "mirror port" to receive
all traffic*





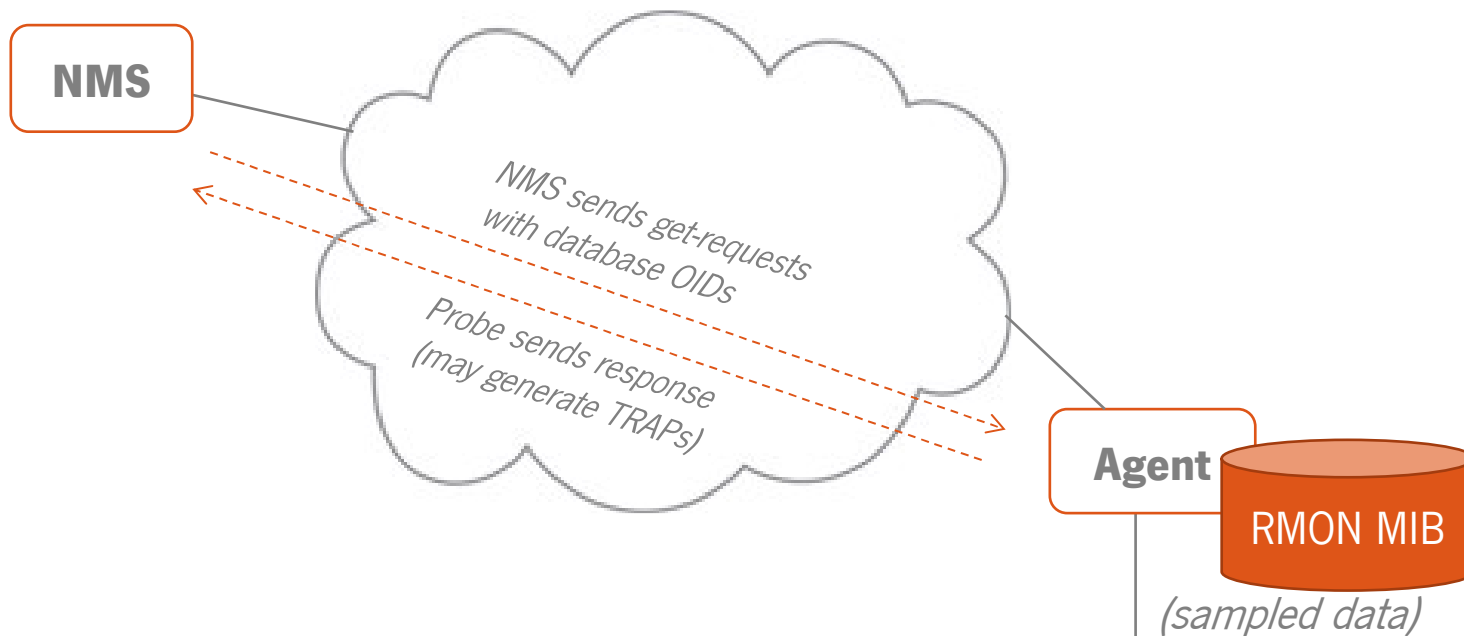
RMON: Goals

- **Get network statistics: #bytes, #packets, #errors, etc...**
- **For:**
 - Long-term data collection
 - Ad-hoc firefighting
- **Use remote network monitoring devices, also called probes or monitors**
 - Remote probes can be standalone devices devoted to management!
 - Or, specific cards in routers/switches dedicated to RMON
 - Or, software packages added to operating systems
- **Admin can strategically place those “monitors” in the entire network**



RMON: Goals

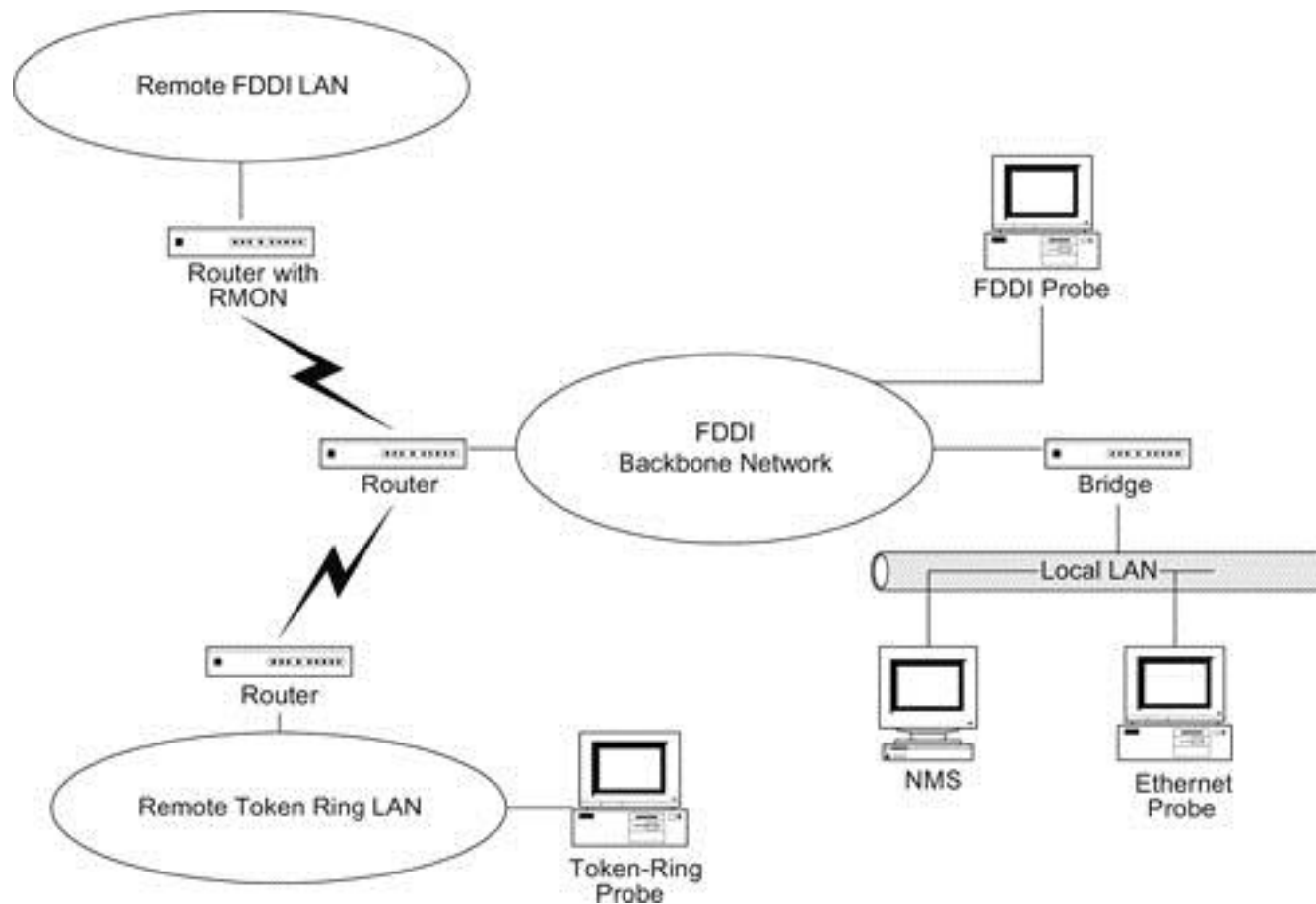
- **RMON probes:**
 - Monitor local network (segment, link, etc)
 - Can be polled by NMS or generate traps on abnormal conditions
 - Can also poll devices in its local network, thus minimizing traffic to NMS



RMON: Goals



- **Network topology with probes (different types)**





RMON: Goals

- **Offline operation**

- Sometimes, by design or by accident, the NMS is not in contact with remote device
- A probe must collect data and perform diagnostics continuously

- **Proactive monitoring**

- The monitor is always available on the onset of any failure
- It can notify the NMS and store historical statistic information to later diagnose the problem

- **Problem detection and reporting**

- The monitor can be configured to recognize conditions, log events and notify NMS

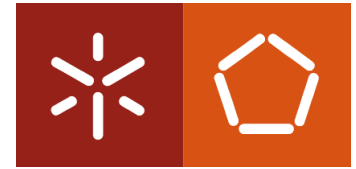
- **Value added data**

- The probe is dedicated to network monitoring and can add value to data, collected locally
- Ex: identify the devices in the network that are generating more traffic

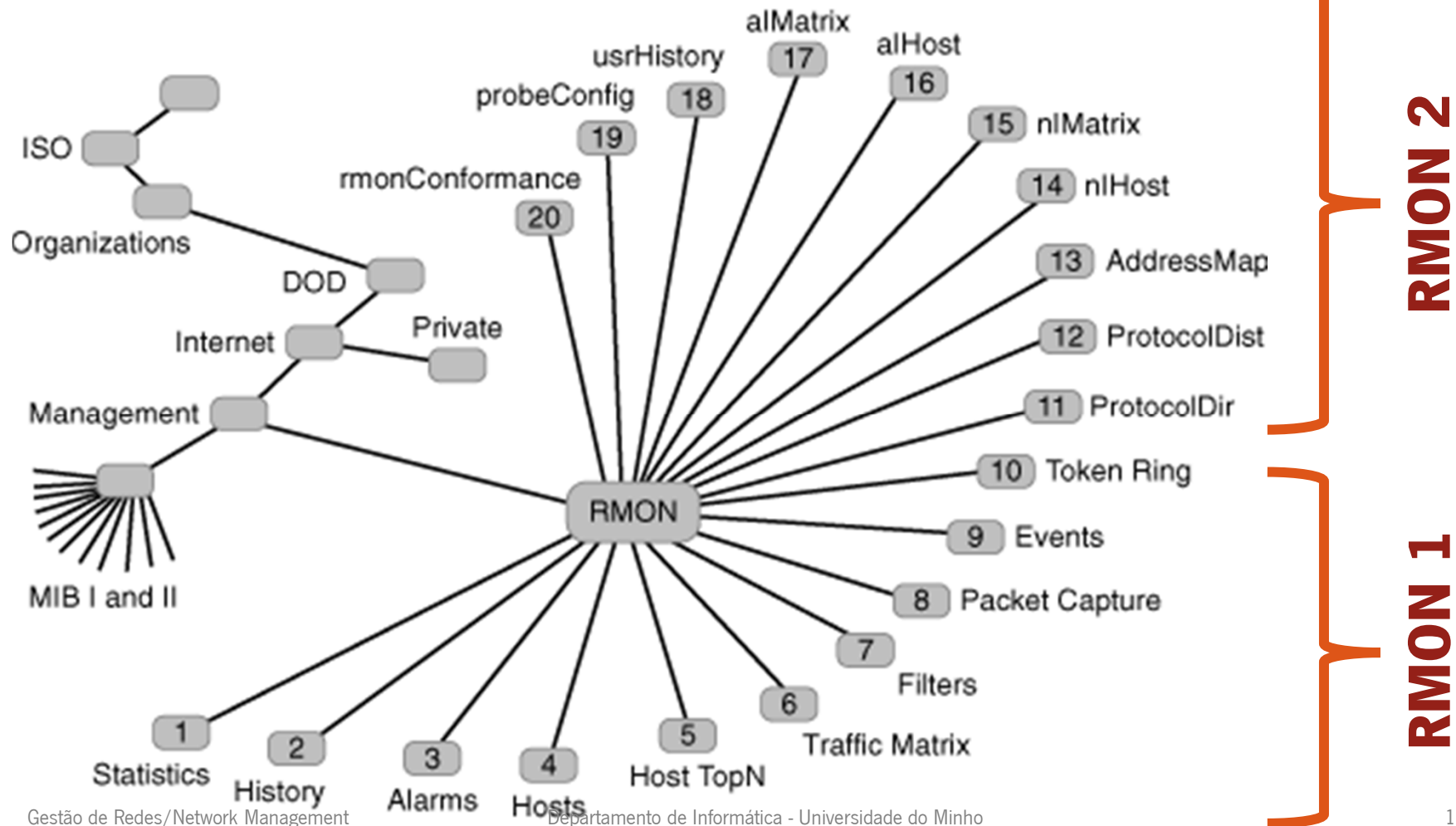
- **Multiple managers**

- More than one manager (NMS) can access data concurrently

RMON: MIB



- **RMON 1, RMON 2, ...**





RMON: MIB Modules

RMON-1	RMON-1 is focused on Layer 2 (link layer statistics)	<i>ethernet statistics, history control, ethernet history, alarm, host, hostTopN, matrix, filter, packet capture, event</i>
TR-RMON	Token Ring extensions to RMON	<i>token-ring statistics, token-ring history, token-ring station, token-ring station order, token-ring station config, routing</i>
RMON-2	RMON-2 extends analysis up to the Application Layer	<i>protocol directory, protocol distribution, address mapping, network layer host, network layer matrix, application layer host, application layer matrix, user history, probe configuration</i>
SMON	Remote Network Monitoring MIB Extensions for Switched Networks	<i>ProbeCapabilities, smonVlanStats, smonPrioStats, dataSourceCaps, portCopyConfig</i>
IFTopN	RMON MIB Extensions for Interface Parameters Monitoring	<i>interfaceTopNObjects</i>
HCRMON	RMON for High Capacity Networks	Adds a <i>mediaIndependentStats</i> group to RMON 1 and 2
TPM	Transport Performance Metrics MIB	<i>tpmCapabilitiesGroup, tpmAggregateReportsGroup, tpmCurrentReportsGroup, tpmExceptionReportsGroup</i>
APM	Application Performance Measurement MIB	<i>application directory, user defined applications, report, transaction, txception, notification groups</i>



RMON: MIB Groups

- **How to differentiate RMON MIB Modules?**
 - At which layer does the MIB take measures?
 - At which layers does the MIB aggregate measurements?

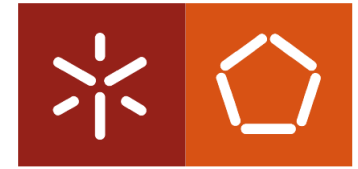
MEASUREMENT LAYERS

	Data Link Layer	Network Layer	Transport Layer	Application Layer
RMON-1	X			
TR-RMON	X			
RMON-2	X			
SMON	X			
IFTopN	X			
HCRMON	X			
APM				X
TPM			X	

AGGREGATION LAYERS

	Data Link Layer	Network Layer	Transport Layer	Application Layer
RMON-1	X			
TR-RMON	X			
RMON-2		X	X	X
SMON	X			
IFTopN	X			
HCRMON	X			
APM		X	X	X
TPM		X	X	X

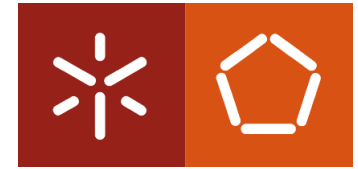
RMON 1



statistics (1)

- We already have "interfaces" group in MIB-II
 - aggregate traffic statistics per interface: #bytes, #pkts, #errors, .. (in/out)
 - But: no local intelligence, highly aggregated link statistics, polling required
- Stats aggregated on per link basis (all traffic from all hosts in link) as aggregate and not by manager from every host on link
- For every monitored LAN segment:
 - Number of packets, bytes, broadcast/multicast packets, CRC errors, length problem, collisions
 - Size histogram: [64, 65-127, 128-255, 256-511, 512-1023, 1024-1518]
 - Similar to interface group, but computed over entire traffic on LAN

RMON 1



statistics (1)

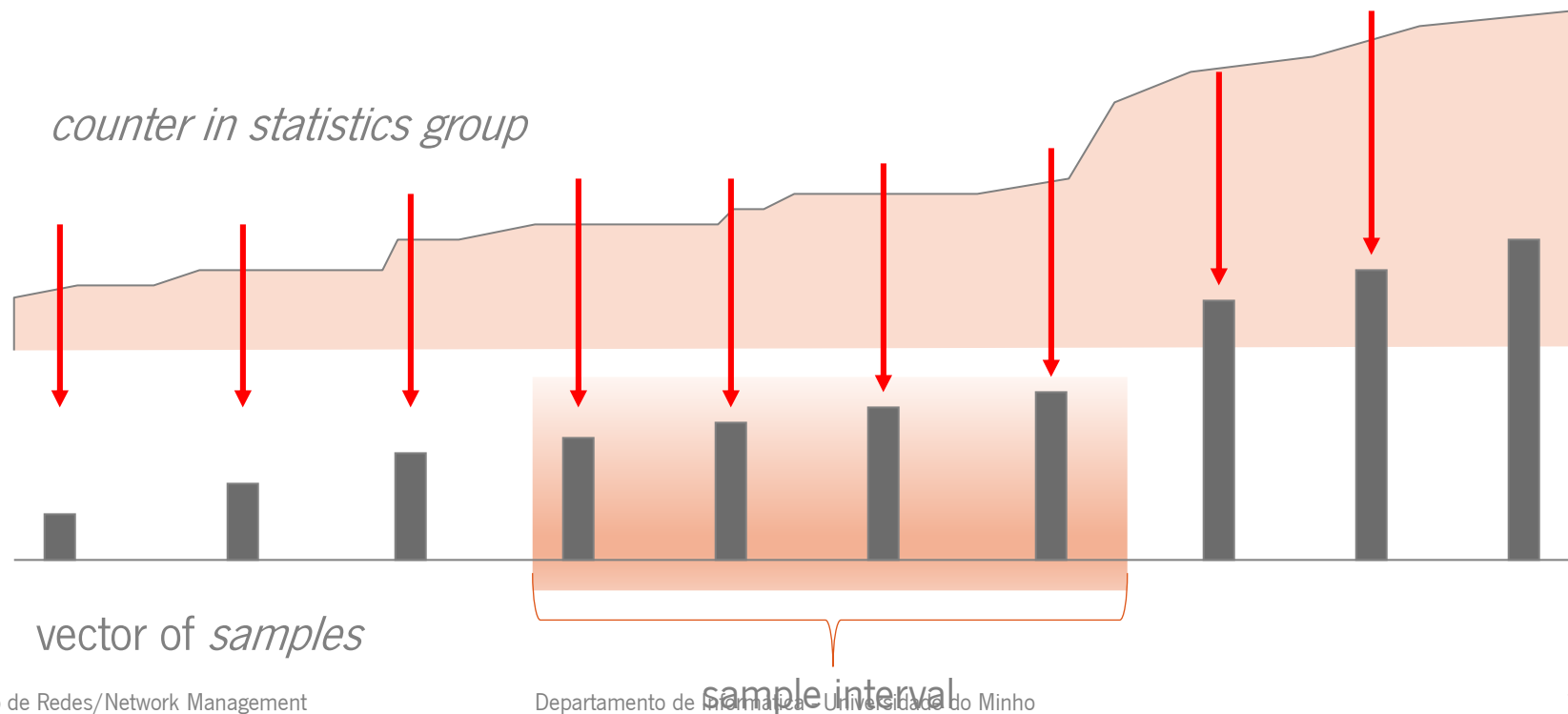
```
EtherStatsEntry ::= SEQUENCE {  
    etherStatsIndex          Integer32,  
    etherStatsDataSource     OBJECT IDENTIFIER  
    etherStatsDropEvents     Counter32,  
    etherStatsOctets         Counter32,  
    etherStatsPkts           Counter32,  
    etherStatsBroadcastPkts  Counter32,  
    etherStatsMulticastPkts  Counter32,  
    etherStatsCRCAlignErrors Counter32,  
    etherStatsUndersizePkts  Counter32,  
    etherStatsOversizePkts   Counter32,  
    etherStatsFragments      Counter32,  
    etherStatsJabbers        Counter32,  
    etherStatsCollisions     Counter32,  
    etherStatsPkts64Octets   Counter32,  
    etherStatsPkts65to127Octets Counter32,  
    etherStatsPkts128to255Octets Counter32,  
    etherStatsPkts256to511Octets Counter32,  
    etherStatsPkts512to1023Octets Counter32,  
    etherStatsPkts1024to1518Octets Counter32,  
    etherStatsOwner          OwnerString,  
    etherStatsStatus         EntryStatus  
}
```

RMON 1



history (2)

- Parameters: sample interval, # buckets
- Similar to statistics, but for time intervals (sliding window)



Control table
(defines source,
interval, owner, ...)

```
HistoryControlEntry ::= SEQUENCE {
    historyControlIndex          Integer32,
    historyControlDataSource     OBJECT IDENTIFIER,
    historyControlBucketsRequested Integer32,
    historyControlBucketsGranted Integer32,
    historyControlInterval       Integer32,
    historyControlOwner           OwnerString,
    historyControlStatus          EntryStatus
}
```

```

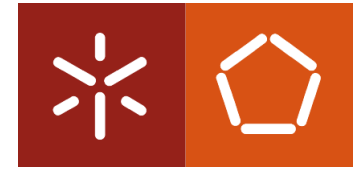
EtherHistoryEntry ::= SEQUENCE {
    etherHistoryIndex
    etherHistorySampleIndex
    etherHistoryIntervalStart
    etherHistoryDropEvents
    etherHistoryOctets
    etherHistoryPkts
    etherHistoryBroadcastPkts
    etherHistoryMulticastPkts
    etherHistoryCRCAlignErrors
    etherHistoryUndersizePkts
    etherHistoryOversizePkts
    etherHistoryFragments
    etherHistoryJabbers
    etherHistoryCollisions
    etherHistoryUtilization
}

```

[illegible]

Data table with samples, up to granted buckets...

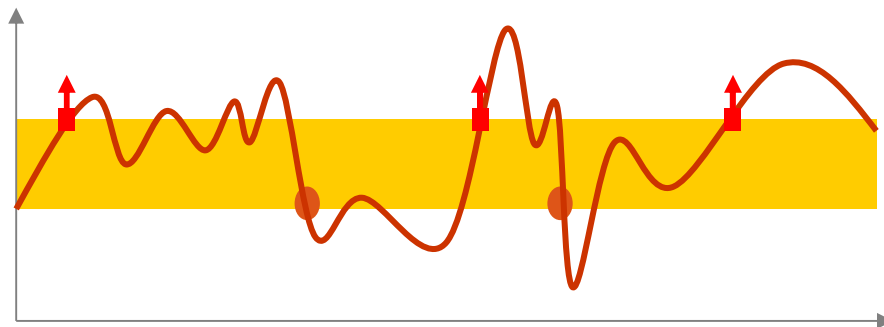
RMON 1



alarm (3)

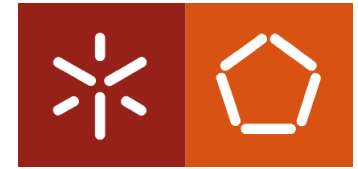
- The alarm table stores configuration entries to define variable, polling interval, and threshold parameters
- The alarm group requires also events group to identify actions
- Define thresholds (rising, falling, or both)
 - absolute: e.g., alarm as soon as 1000 errors have accumulated
 - delta: e.g., alarm if error rate over an interval $> 1/\text{sec}$

- **Limiting alarm overhead: hysteresis**



```
AlarmEntry ::= SEQUENCE {  
    alarmIndex                Integer32,  
    alarmInterval             Integer32,  
    alarmVariable             OBJECT IDENTIFIER,  
    alarmSampleType           INTEGER,  
    alarmValue                Integer32,  
    alarmStartupAlarm         INTEGER,  
    alarmRisingThreshold      Integer32,  
    alarmFallingThreshold     Integer32,  
    alarmRisingEventIndex     Integer32,  
    alarmFallingEventIndex    Integer32,  
    alarmOwner                OwnerString,  
    alarmStatus               EntryStatus  
}
```

RMON 1



host (4)

- Aggregate statistics per host (for each host discovered in the network): pkts in/out, bytes in/out, errors, broadcast/multicast pkts

hostTopN (5)

- Creates a report of the top N hosts, according to several metrics, defined in control table

matrix (6)

- Matrix of traffic statistics for each source and destination

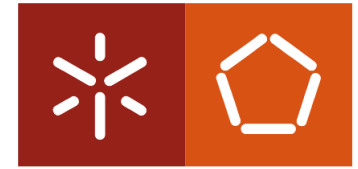
Control Table

```
MatrixControlEntry ::= SEQUENCE {  
    matrixControlIndex      Integer32,  
    matrixControlDataSource OBJECT IDENTIFIER,  
    matrixControlTableSize  Integer32,  
    matrixControlLastDeleteTime TimeTicks,  
    matrixControlOwner      OwnerString,  
    matrixControlStatus     EntryStatus  
}
```

Data Table

```
MatrixSDEntry ::= SEQUENCE {  
    matrixSDSourceAddress  OCTET STRING,  
    matrixSDDestAddress   OCTET STRING,  
    matrixSDIndex          Integer32,  
    matrixSDPkts           Counter32,  
    matrixSDOctets         Counter32,  
    matrixSDErrors         Counter32  
}
```

RMON 1



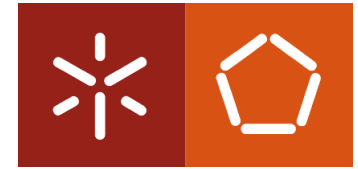
filter (7)

- allows packets to be captured with an arbitrary filter expression
- a "channel" is formed with the stream of packets that match the filter
- The filter is defined as boolean functions on bit patterns and packet status

capture (8)

- Stores the packets captured according to a filter
- The bufferControlTable controls the captured packets output from a channel that is associated with it.
- The captured packets are placed in entries in the captureBufferTableFilter

RMON 1



event (9)

- The Event group controls the generation and notification of events from the device
- Each eventEntry may optionally specify that a log entry be created on its behalf whenever the event occurs.

```
EventEntry ::= SEQUENCE {  
    eventIndex          Integer32,  
    eventDescription    DisplayString,  
    eventType           INTEGER,  
    eventCommunity      OCTET STRING,  
    eventLastTimeSent   TimeTicks,  
    eventOwner          OwnerString,  
    eventStatus         EntryStatus  
}
```

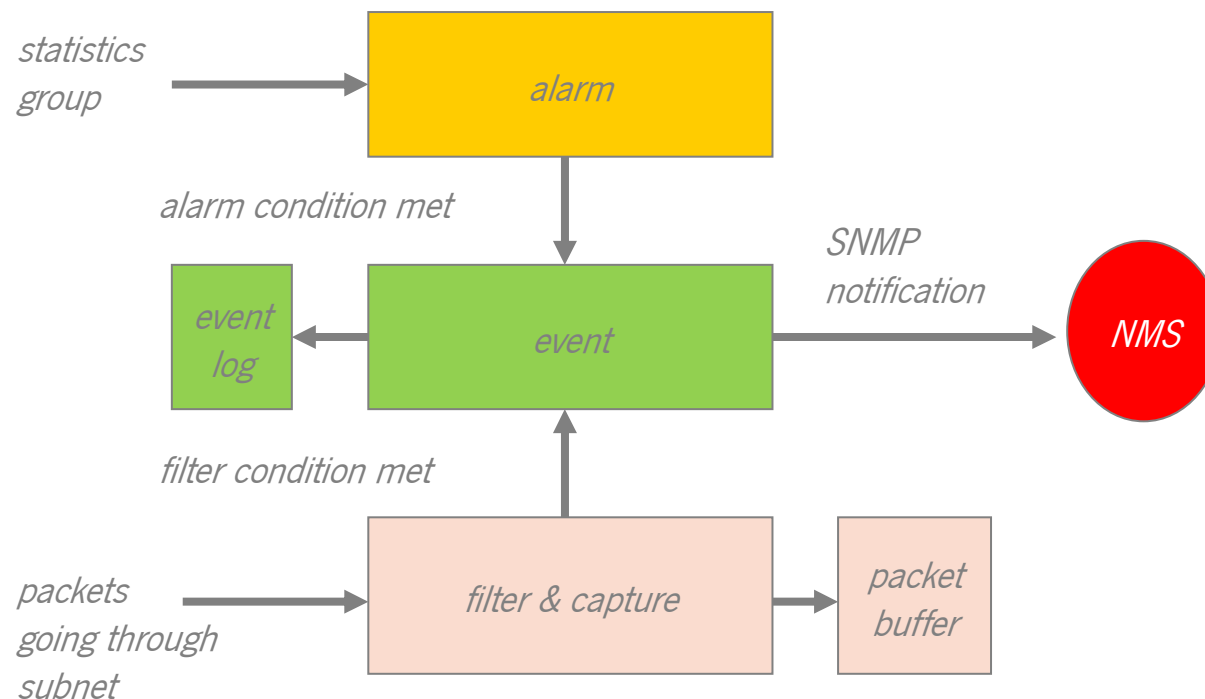
```
LogEntry ::= SEQUENCE {  
    logEventIndex      Integer32,  
    logIndex           Integer32,  
    logTime            TimeTicks,  
    logDescription     DisplayString  
}
```



RMON 1: overview

- There are passive metrics and active metrics
- Active metrics require more resources: dedicated probes

RMON ACTIVE METRICS



RMON 2



- **Extends monitoring to upper layers**

protocolDir	Lists the protocols supported by the RMON probe. Allows the addition, deletion, and configuration of entries in this list.
protocolDist	Shows statistics (number of octets and packets) on a per-protocol basis.
addressMap	Lists MAC address to network address bindings and the interface they were last seen on.
nlHost	Network Layer Host group. Counts the amount of traffic sent from and to each discovered network address.
nlMatrix	Network Layer Matrix group. Counts the amount of traffic sent between each pair of discovered network addresses.
alHost	Application Layer Host group. Counts the amount of traffic, per protocol, sent from and to each discovered network address.
alMatrix	Application Layer Matrix group. Counts the amount of traffic, per protocol, sent between each pair of discovered network addresses.
usrHistory	User History group. Combines mechanisms from the Alarms and History groups to provide history collection based on user-defined criteria.
probeConfig	Controls the configuration of RMON features.
rmonConformance	Describes conformance requirements.